

Programowanie obiektowe

Wykład 9

Wyjątki

Rodzaje błędów w oprogramowaniu

- Błędy czasu kompilacji (*compile-time errors*)
 - Wykrywane przez kompilator
 - Uniemożliwiają uruchomienie programu
 - **Przykłady:** brak średnika, zła deklaracja typu, niezdefiniowana zmienna
- Błędy czasu wykonywania (*run-time errors*)
 - Występują podczas działania programu
 - Mogą prowadzić do awarii lub nieoczekiwanych rezultatów
 - Mogą być trudne do przewidzenia (np. zależą od danych użytkownika)
 - Mogą pochodzić z zewnątrz (system operacyjny, API, sprzęt)
 - **Przykłady:** dzielenie przez zero, przekroczenie zakresu tablicy, błąd odczytu z pliku
- Błędy logiczne (*logic errors*)
 - Program się kompiluje i działa, ale nie spełnia założeń programisty
 - Najtrudniejsze do wykrycia
 - Mogą być efektem nieuwagi, braku wiedzy, złego projektu
 - **Przykłady:** zła formuła matematyczna, zła kolejność warunków, niewłaściwe porównania

Błędy czasu wykonania

- Reagowanie na błędy czasu wykonania może dotyczyć:

Błądów wykrytych po stronie *naszego kodu*

```
public class BankAccount
{
    private double balance;

    public void Withdraw(double amount)
    {
        if (amount > balance)
            ...

        balance -= amount;
    }
}
```

- Użytkownik próbuje wypłacić więcej środków, niż posiada na koncie
- Powinniśmy ostrzec zleceniodawcę (np. klienta API lub użytkownika)

Błądów po stronie *zleceniobiorcy* (zewnętrznego komponentu, biblioteki)

```
var processor = new CsvProcessor();
processor.processFile("dane.csv");
```

- Chcemy wiedzieć, czy zlecona operacja zakończyła się powodzeniem.
- Jeśli wewnątrz biblioteki wystąpił błąd, powinniśmy zostać o tym poinformowani.

Regowanie na błędy czasu wykonania

- Jak możemy informować o wystąpieniu błędu?

Wypisywanie komunikatu

```
if (amount > balance)
    Console.WriteLine("Brak środków.");
```

- Informacja tylko dla człowieka, nie dla kodu – program nadal nie wie, że coś poszło nie tak i nie może zareagować.

```
var account = new BankAccount();
account.Withdraw(1200.0);
// ???
```

- Użytkownik też nie wie, co zrobić – komunikat nie dostarcza możliwości rozwiązania problemu.

Zwracanie wartości

```
if (amount > balance)
    return false;
```

- Zmusza do sprawdzania wyniku po każdej operacji, tworząc wiele nadmiarowego kodu.

```
var account = new BankAccount();
if(!account.Withdraw(1200.0)) {
    // !!!
}
```

- Zwiększa ryzyko pominięcia obsługi błędu, składnia języka (C++, Java, C#) nie zmusza do sprawdzenia zwracanej wartości.

Wyjątki

Koncepcja wyjątków (*Exceptions*) – wprowadzenie

- **Definicja:** Wyjątek (ang. *exception*) to zdarzenie sygnalizujące wystąpienie nieoczekiwanej lub nieprawidłowej sytuacji w trakcie działania programu, która uniemożliwia kontynuację normalnego przebiegu wykonania.
- **Cel:**
 - Oddzielenie logiki obsługi błędów od logiki głównej.
 - Umożliwienie propagacji informacji o błędzie do wyższych poziomów aplikacji, gdzie mogą być podjęte działania naprawcze lub można zdecydować o zakończeniu programu.
- **Zalety:**
 - Czytelniejszy i bardziej spójny kod (unikanie konieczności kontroli błędów po każdej operacji).
 - Możliwość zgrupowania obsługi w jednym lub kilku specjalizowanych blokach.
- W programie obiektowym, **wyjątek** to również obiekt reprezentujący sytuację błędną, nietypową lub wyjątkową, która wystąpiła w czasie działania programu i wymaga specjalnej obsługi.
 - **Wyjątek** może być **rzucony** (zgłoszony, podniesiony) w jednej części kodu, a **złapany** i obsługiwany w innej.

Rzucanie wyjątku – `throw`

Słowo kluczowe `throw` służy do **rzucania wyjątku**, czyli przerywania standardowego przepływu programu i przekazania błędu do miejsca, które go obsłuży.

- Użycie `throw` tworzy wyjątek i przekazuje go dalej w strukturze wywołań funkcji (tzw. *stack unwinding*).

Ogólna składnia

```
public class BankAccount
{
    private double balance;

    public void Withdraw(double amount)
    {
        if (amount > balance)
            throw wyjątek;

        balance -= amount;
    }
}
```

- Program natychmiast przerywa aktualne wykonanie.
- Poszukiwany jest najbliższy blok, który potrafi obsłużyć dany typ wyjątku.
- Jeśli nie zostanie on odnaleziony, program zakończy działanie.

throw w C++

- Można rzucać:
 - Dowolne typy (`int` , `char*` , `std::string` , obiekty własnych klas...)
 - Zalecane: obiekty klas dziedziczących po `std::exception`

Przykład:

```
if (balance < amount)
    throw std::logic_error("Zbyt mało środków na koncie");
```

throw w Javie

- Można rzucać tylko obiekty klas dziedziczących po `Throwable` (np. `Exception`, `RuntimeException`, `Error`).

```
throw new IllegalArgumentException("Nieprawidłowy argument");
```

- W przypadku **checked exceptions** (wyjątki sprawdzane) należy zadeklarować wyjątek w sygnaturze metody (`throws`)

```
public void readFile() throws IOException {  
    ...  
    throw new IOException("Brak dostępu do pliku");  
    ...  
}
```

- Metoda nie ma prawa wyrzucić *checked exception*, jeśli przed nim nie ostrzeże (nie wyspecyfikuje go) – próba wyrzucenia weryfikowanego wyjątku bez specyfikacji powoduje błąd kompilacji.
- Unchecked exceptions** (wyjątki niesprawdzane, np. `NullPointerException`, `IllegalArgumentException`) nie wymagają deklaracji `throws`.

throw w C#

- Można rzucać tylko obiekty klas dziedziczących po `System.Exception`.
- Nie wymaga deklarowania wyjątków (brak *checked exceptions*).

```
throw new ArgumentOutOfRangeException("param", "Nieprawidłowy zakres");
```

Dobre praktyki rzucania wyjątków

- Wyjątek należy rzucać tylko w sytuacji naprawdę wyjątkowej
 - Nie powinno się używać `throw` do normalnej logiki (np. do sterowania przepływem programu).
 - Wyjątki powinny sygnalizować **błąd**, którego nie da się przewidzieć lub obsłużyć lokalnie.
- Wyjątki należy rzucać z komunikatem

```
throw new IllegalArgumentException("ID użytkownika nie może być puste");
```

- Komunikat powinien być **konkretny, jednoznaczny i pomocny** podczas debugowania.
- Błędem jest rzucanie pustych wyjątków

```
throw new RuntimeException();
```

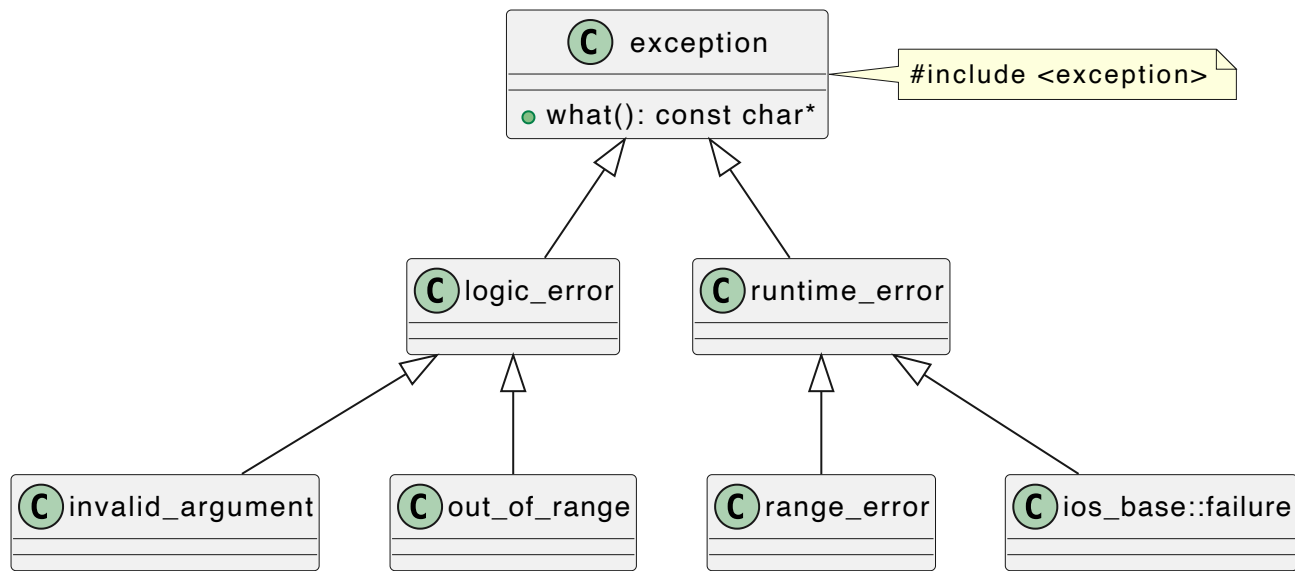
- Nie należy używać `throw` do przerywania pętli, warunków, sprawdzania wartości itp.
- Poprawne porzypadki użycia wyjątków:
 - Sygnał o błędzie krytycznym (np. brak zasobu, dzielenie przez 0)
 - Walidacja danych wejściowych
 - Naruszenie logiki biznesowej (np. brak autoryzacji)
 - Ochrona przed nieprawidłowym stanem programu

Hierarchia wyjątków - C++

- Podstawowa klasa bazowa `std::exception` zawiera wirtualną metodę `what()`, zwracającą komunikat tekstowy (można ją nadpisać we własnych klasach).
- Wybrane klasy dziedziczące:
 - Standardowe wyjątki logiczne (`std::logic_error`), np.
 - `std::invalid_argument` - niepoprawny parametr funkcji
 - `std::out_of_range` - indeks poza zakresem
 - Wyjątki wykonania (`std::runtime_error`), np.
 - `std::range_error` - błąd zakresu: wyniku obliczeń nie da się przedstawić w docelowym typie
 - `std::ios_base::failure` - błędy strumieni I/O
- Typy podstawowe (`int`, `char*`) nie są zalecane - nie pozwalają na jednolitą obsługę
- **Zalecane:** używanie klas z `std::exception` lub tworzenie własnych wyjątków dziedziczących po niej.

```
class MyException : public std::exception {
public:
    const char* what() const noexcept override {
        return "Wystąpił błąd aplikacji";
    }
};
```

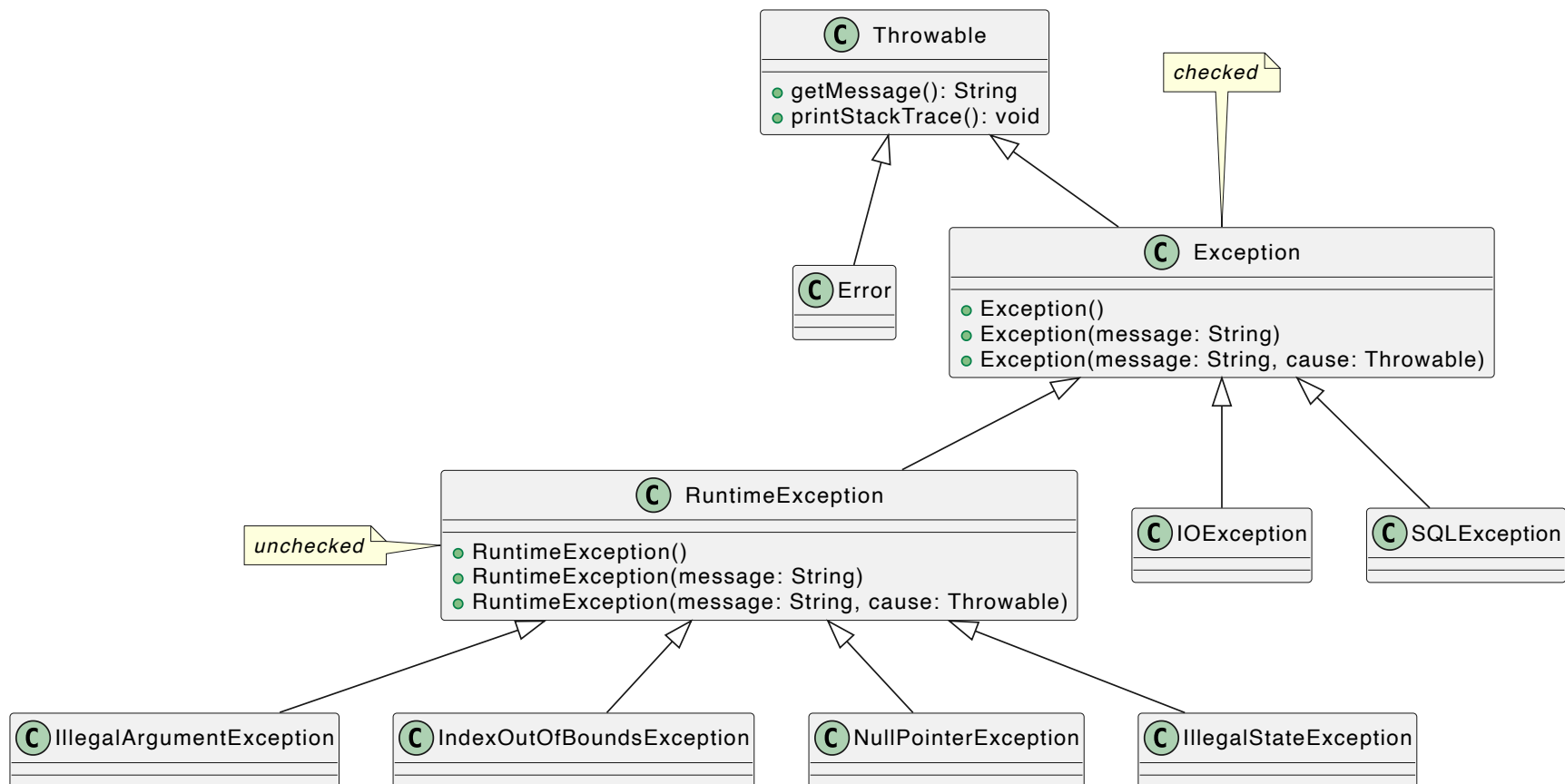
Hierarchia wyjątków w C++



Hierarchia wyjątków - Java

- Klasa bazowa to `java.lang.Throwable`. Dziedziczą z niej:
 - `Error` - krytyczne błędy JVM (nie należy ich obsługiwać)
 - `Exception` - typowe wyjątki aplikacyjne
- **Checked exceptions** (kompilator wymaga ich obsłużenia), np.:
 - `IOException` - błędy plików/strumieni
- **Unchecked exceptions** (dziedziczą po `RuntimeException`), np.:
 - `NullPointerException` - operacja na nullu
 - `IllegalArgumentException` - nieprawidłowy parametr
 - `IndexOutOfBoundsException` - indeks poza zakresem
 - `ArithmeticException` - dzielenie przez zero itp.
 - `IllegalStateException` - obiekt w nieprawidłowym stanie
 - `UnsupportedOperationException` - metoda niezaimplementowana
- **Zalecane:**
 - Rzucanie **checked exceptions** tam, gdzie wywołujący powinien reagować.
 - Tworzenie własnych wyjątków dziedziczących po `Exception` lub `RuntimeException` zgodnie z kontekstem.

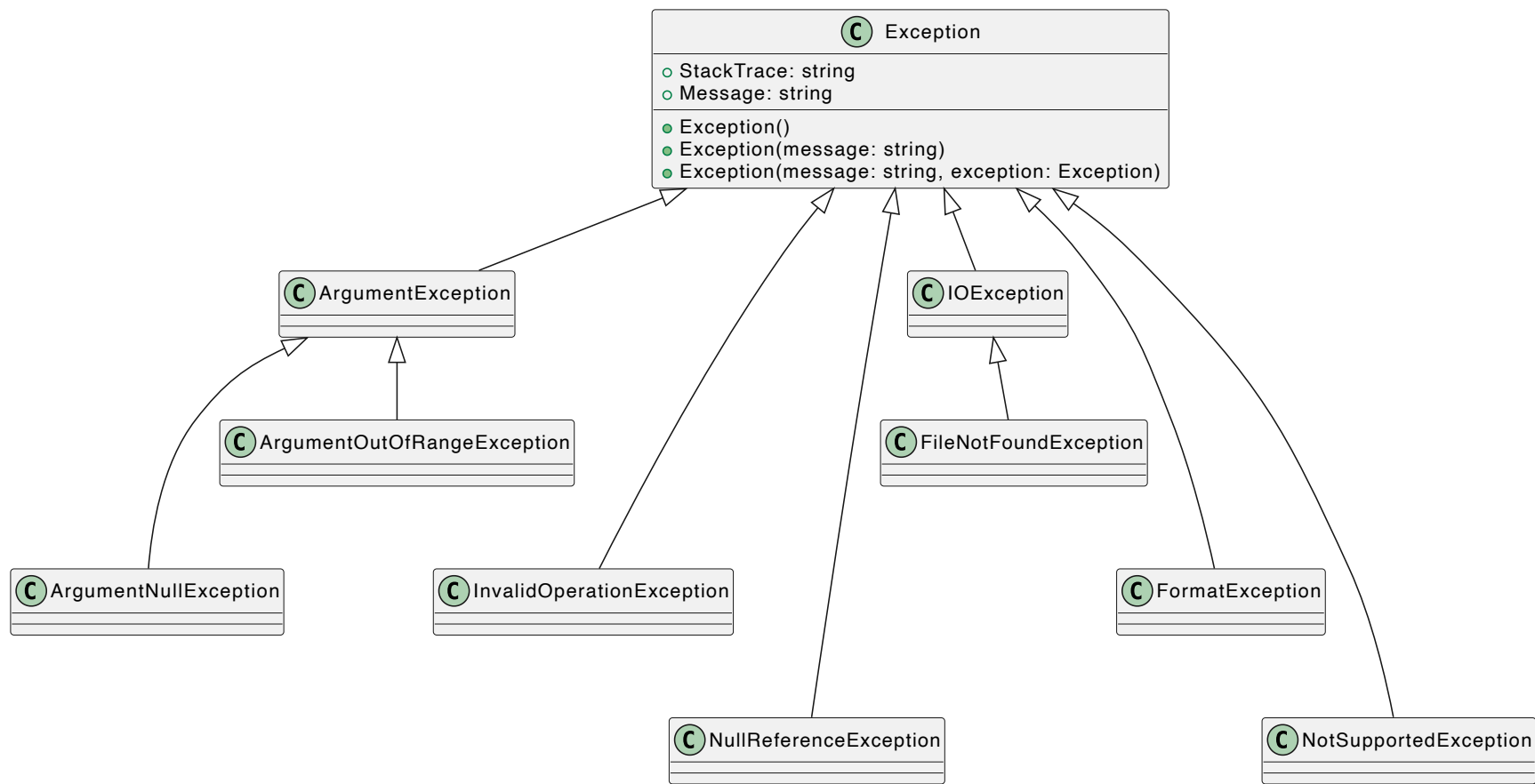
Hierarchia wyjątków w Javie



Hierarchia wyjątków - C#

- Klasa bazowa to `System.Exception`.
- Najczęściej używane klasy wyjątków:
 - `ArgumentException` - ogólny błąd argumentu
 - `ArgumentNullException` - wartość `null`
 - `ArgumentOutOfRangeException` - niewłaściwy zakres argumentu
 - `InvalidOperationException` - nieprawidłowa operacja w danym stanie
 - `NullReferenceException` - odwołanie do `null`
 - `IndexOutOfRangeException` - błędny indeks
 - `IOException` - błędy I/O, np. `FileNotFoundException`, `DirectoryNotFoundException`
 - `NotSupportedException` - metoda nieobsługiwana
 - `FormatException` - błędny format danych
- Systemowe wyjątki niskiego poziomu (rzadko obsługiwane): `OutOfMemoryException`, `StackOverflowException`, `AccessViolationException`
- **Zalecane:** tworzenie własnych wyjątków dziedziczących po `Exception` lub po konkretnej klasie.

Hierarchia wyjątków w C#



Własna hierarchia wyjątków

- Dobrą praktyką jest tworzenie **własnych klas wyjątków** – szczególnie tam, gdzie potrzebujemy czytelnej, kontrolowanej i specjalistycznej obsługi błędów domenowych.

Zamiast:

```
throw new Exception("Zły ID");
```

Lepiej:

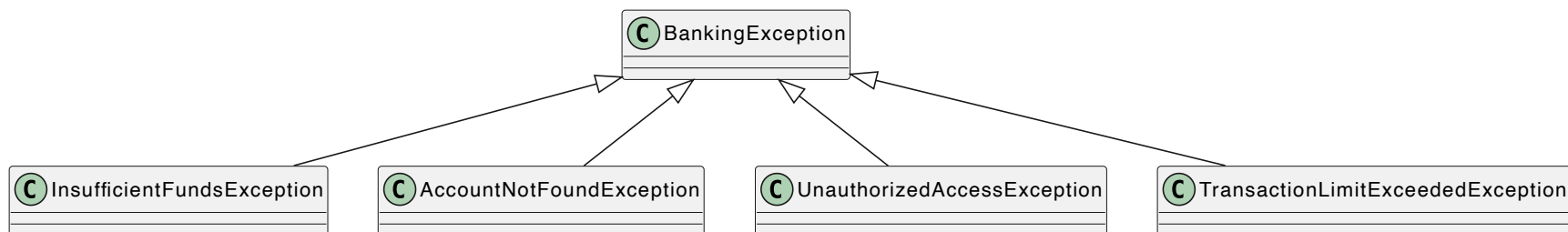
```
throw new InvalidClientException("Klient ID: 123");
```

Jak tworzyć własne wyjątki?

- C++ – klasa bazowa `std::exception`
- Java – `Exception` lub `RuntimeException`
- C# – `System.Exception`

Własna hierarchia wyjątków - przykład

Założmy przykładową aplikację bankową.



- **Wyraźne nazwy** - pozwalają od razu zrozumieć, co się stało (`InsufficientFundsException` , `AccountNotFoundException`)
- **Oddzielają wyjątki techniczne od domenowych** - np. błąd I/O vs. nieautoryzowany dostęp do konta (`UnauthorizedAccessException`)
- **Pozwalają na spójną obsługę klas błędów** - wspólny `BankingException` jako baza
- **Mogą zawierać dodatkowe dane** - np. identyfikator klienta, numer konta
- Uwaga na **balans**:
 - zbyt wiele wyjątków = nadmiarowy kod,
 - zbyt mało = brak czytelności.

Własna hierarchia wyjątków – przykład (Java)

- Klasa bazowa

```
public class BankingException extends RuntimeException {  
    public BankingException(String message) {  
        super(message);  
    }  
}
```

- Konkretna klasa wyjątku

```
public class InsufficientFundsException extends BankingException {  
    private final String accountId;  
    private final double attemptedAmount;  
    private final double availableBalance;  
  
    public InsufficientFundsException(String accountId, double attemptedAmount,  
                                     double availableBalance) {  
        super("Konto: " + accountId + ", próba wypłaty: " + attemptedAmount +  
              ", dostępne środki: " + availableBalance);  
        this.accountId = accountId;  
        this.attemptedAmount = attemptedAmount;  
        this.availableBalance = availableBalance;  
    }  
  
    public String getAccountId() { return accountId; }  
    public double getAttemptedAmount() { return attemptedAmount; }
```

Dobre i złe praktyki użycia wyjątków

- Należy używać odpowiedniego typu wyjątku:
 - Wyjątków standardowych (`IllegalArgumentException` , `IOException` itd.) tam, gdzie pasują.
 - **Własnych wyjątków domenowych** (`InsufficientFundsException`), jeśli kontekst tego wymaga.
- **Złą praktyką** jest rzucanie ogólnego `Exception` lub `Throwable` – utrudnia to selektywną obsługę wyjątków

```
throw new Exception("Coś poszło nie tak"); // Nie wiadomo co konkretnie
```

Specyfikowanie wyjątków - Java

- W Javie, jeśli metoda może rzucić wyjątek typu *checked* (sprawdzany), musi to zadeklarować w swojej sygnaturze za pomocą słowa `throws`.
- To informuje kompilator (i programistę), że wywołanie tej metody wymaga obsługi wskazanych wyjątków.

```
public void loadConfig() throws IOException {  
    Files.readAllLines(Path.of("config.txt")); // metoda rzuca IOException  
}
```

- Specyfikowanie dotyczy tylko wyjątków *checked*, czyli dziedziczących bezpośrednio po `Exception`, ale nie po `RuntimeException` - te nie muszą być deklarowane ani obsługiwane jawnie.
- Można zadeklarować wiele wyjątków:

```
public void parseData() throws IOException, ParseException {  
    // ...  
}
```

- **Plus:** zmusza programistę do świadomej obsługi błędów.
- **Minus:** może prowadzić do nadmiernego kodu obsługi błędów (*exception pollution*).
- **C#** - choć język tego nie wymusza, dobrą praktyką jest dokumentowanie, jakie wyjątki mogą być rzucone.
- **C++** - przestarzała i niezalecana składnia pozwalała specyfikować wyjątki, obecnie używa się `noexcept` aby zadeklarować że funkcja nie rzuca wyjątków

Konsekwencje rzucenia wyjątku

- W momencie, gdy zostaje **rzucony wyjątek** (i nie zostanie on natychmiast obsłużony w aktualnym bloku kodu), następuje **zwijanie stosu** (*stack unwinding*).
- To **automatyczne cofanie się w górę stosu wywołań funkcji/metod**, w poszukiwaniu odpowiedniego bloku, który potrafi obsłużyć rzucony wyjątek.
- W trakcie tego procesu:
 - Zwalniane są wszystkie zasoby lokalne (zmienne, obiekty, uchwyt).
 - W C++ wywoływane są **destruktory** obiektów z każdego opuszczanego zakresu (w przykładzie - `Logger`).
 - Program kończy działanie, jeśli wyjątek nie zostanie nigdzie obsłużony.

Przykład (C++)

```
class Logger {
public:
    Logger() { cout << "start\n"; }
    ~Logger() { cout << "koniec\n"; }
};

void withdrawFunds(...) {
    if (amount > balance)
        throw runtime_error("Brak środków");
    // wykonanie operacji
}

void processTransaction(...) {
    Logger log;
    withdrawFunds(...);
    // jakieś dalsze operacje
}

int main() {
    // jakieś wcześniejsze operacje
    processTransaction(...);
    // jakieś dalsze operacje
}
```

Łapanie wyjątków

- Do **przechwytywania i obsługi wyjątków** służy konstrukcja `try-catch`.
- Pozwala ona na kontrolowane reagowanie na błędy zgłoszone przez `throw` bez przerywania działania całej aplikacji.

```
try {  
    // kod mogący rzucić wyjątek  
} catch {  
    // obsługa wyjątku  
}
```

- W bloku `try` umieszcza się kod, który może spowodować wyrzucenie wyjątku
- Pułapka `catch` łapie wyjątki z powyższego bloku
- Po pułapce następuje kod obsługi wyjątku (*exception handler*)
- Złapany (obsłużony) wyjątek kończy swoją „wędrowkę” po modułach

Posługiwanie się try-catch

- Blok `try` zawiera kod, który może rzucić wyjątek.
- Jeśli w `try` wystąpi wyjątek, dalszy kod jest pomijany, a sterowanie przechodzi do pierwszego pasującego bloku `catch`.
- Jeśli `catch` obsłuży wyjątek, program kontynuuje działanie po bloku `catch`.
- Jeżeli brak pasującego `catch`, stos wywołań jest zwijany (*stack unwinding*), a wyjątek przekazywany wyżej (**propagacja wyjątku**).

Propagacja wyjątków (*exception propagation*) to proces, w którym nieobsłużony wyjątek jest przekazywany w górę stosu wywołań funkcji/metod, aż do momentu, gdy zostanie obsłużony (`catch`) lub doprowadzi do przerwania programu.

```
void withdrawFunds(...) {  
    if (amount > balance)  
        throw new Exception("Brak środków");  
  
    // jeśli zostanie rzucony wyjątek,  
    // poniższe linijki nie wykonają się  
  
    balance -= amount;  
}  
  
try {  
    // kod mogący rzucić wyjątek  
    beginTransaction();  
    withdrawFunds(1200.0);  
  
    // poniższe linijki zostaną pominięte  
    recordTransaction(-1200.0);  
    commitTransaction();  
}  
catch {  
    // obsługa wyjątku  
    // transakcja nie powiodła się  
}
```

Wiele bloków catch

- Można zdefiniować wiele bloków `catch` - dla różnych typów wyjątków.
- Kolejność `catch` ma znaczenie - dopasowanie działa od góry do dołu.

```
BankService bank = new BankService();

try
{
    bank.withdraw("ACC404", 500.0);
}
catch (AccountNotFoundException e)
{
    System.out.println("Konto nie istnieje: " + e);
}
catch (InsufficientFundsException e)
{
    System.out.println("Za mało środków na koncie: " + e);
}
catch (BankException)
{
    System.out.println("Wystąpił błąd banku.");
}
```

Kolejność bloków `catch`

- Gdy łapiemy wyjątki powiązane między sobą **dziedziczeniem**, znaczenie ma kolejność pułapek `catch`
- Należy umieszczać wyjątki bardziej specyficzne przed ogólnymi

```
try {  
    konto.transfer("ACC100", "ACC200", 10000.0);  
} catch (InsufficientFundsException e) { // Brak środków  
    // ...  
} catch (BankOperationException e) {      // Inny błąd bankowy  
    // ...  
} catch (Exception e) {                  // Dowolny wyjątek  
    // ...  
}
```

- Blok `BankOperationException` łapie wyjątki klasy `BankOperationException` oraz wszystkich klas dziedziczących z niej.
- Niewłaściwa kolejność sprawi, że niektóre bloki staną się nieosiągalne

```
try {  
    // ...  
} catch (BankOperationException e) {      // ogólny wyjątek bankowy – łapie wszystko  
    // ...  
} catch (InsufficientFundsException e) { // blok nigdy nieosiągalny  
    // ...  
}
```

Dobre i złe praktyki łapania wyjątków

- Należy łapać tylko te wyjątki, które znamy i jesteśmy w stanie poprawnie obsłużyć. Nie powinniśmy łapać wyjątków „na wszelki wypadek”, a jedynie wtedy, gdy możemy zareagować w kontrolowany sposób.

```
try {  
    transferMoney();  
} catch (InsufficientFundsException e) {  
    showError("Za mało środków.");  
}
```

- Nie należy łapać zbyt ogólnych wyjątków (`Exception` , `Throwable` , `RuntimeException`). To utrudnia rozpoznanie co się stało i może powodować łapanie wyjątków, których nie jesteśmy w stanie obsłużyć (`OutOfMemoryError`).

```
catch (Exception e) { ... }
```

- Lepszym rozwiązaniem jest rozróżnianie typów błędów, co pozwala reagować na nie inaczej:

```
try {  
    processRequest();  
} catch (AuthenticationException e) {  
    showError("Nieautoryzowany dostęp.");  
} catch (TimeoutException e) {  
    showError("Przekroczono limit czasu.");  
}
```

Błędy użycia mechanizmu wyjątków

✗ Rzucanie i łapanie wyjątku w tym samym bloku

- Ręcznie rzucamy wyjątek, a następnie łapiemy go w tym samym bloku kodu.

```
try {  
    if (amount < 0) {  
        throw new IllegalArgumentException("Kwota nie może być ujemna");  
    }  
} catch (IllegalArgumentException e) {  
    System.out.println("Podano złą kwotę: " + e.getMessage());  
}
```

- Rzucanie wyjątku tutaj **nie ma sensu** – skoro mamy już warunek `if`, wystarczy go obsłużyć:

```
if (amount < 0) {  
    System.out.println("Podano złą kwotę: Kwota nie może być ujemna");  
}
```

- Ewentualnie możemy przerwać aktualną operację i przekazać kontrolę dalej:

```
if (amount < 0) {  
    throw new IllegalArgumentException("Kwota nie może być ujemna");  
}
```

Błędy użycia mechanizmu wyjątków

✗ Używanie `try-catch` jako zamiennika `if-else` (kontrola przepływu)

- Próbujemy wykonać operację w `try`, a jeśli się nie powiedzie, przechodzimy do alternatywnej ścieżki.

```
try {  
    // próbujemy dokonać wpłaty na konto  
    findAccount("ACC123").deposit(100.0);  
} catch (NullPointerException e) {  
    // jeśli się nie powiedzie - zakładamy nowe  
    createAccount("ACC123").deposit(100.0);  
}
```

- To **niewłaściwy** sposób wykorzystania mechanizmu wyjątków. Użyliśmy wyjątków zamiast warunku logicznego.
- Wyjątków powinno się używać tylko do **rzadkich, nieoczekiwanych sytuacji**.
- `NullPointerException` to efekt **błędu programisty**, a nie część normalnego przepływu programu.
- Poprawna wersja:

```
Account acc = findAccount("ACC123");  
if (acc == null) {  
    acc = createAccount("ACC123");  
}  
acc.withdraw(100.0);
```

catch w C++

- Można łapać wyjątki różnych typów.
- Wyjątki należy łapać przez referencję do `const`

```
try {  
    // kod mogący rzucić wyjątek  
}  
catch (const networkIOException& e) {  
    std::cerr << "Błąd połączenia: " << e.what() << std::endl;  
}  
catch (const bankException& e) {  
    std::cerr << "Błąd banku: " << e.what() << std::endl;  
}  
catch (...) {  
    std::cerr << "Nieoczekiwany błąd." << std::endl;  
}
```

- `catch (...)` – przechwytuje wszystkie typy (niebezpieczne i niezalecane).
- Informacje o błędach należy wypisywać na `std::cerr` a nie `std::cout` :
 - `std::cout` – standardowe wyjście (dla danych programu, wyników)
 - `std::cerr` – standardowe wyjście błędów (komunikaty o błędach) – narzędzia zewnętrzne mogą je przekierować do osobnego terminalu lub pliku

catch w Javie

- Jeśli nie obsługujemy wyjątku sprawdzanego (typu *checked*), należy zadeklarować jego rzucenie dalej (`throws`).

```
public void withdraw(double amount)
    throws InvalidAmountException, InsufficientFundsException { ... }

public void processWithdrawal(String accountId, double amount)
    throws InvalidAmountException, AccountNotFoundException {
    try {
        Account account = getAccount(accountId);
        account.withdraw(amount);
    } catch (InsufficientFundsException e) {
        // łapiemy tylko InsufficientFundsException, resztę puszczamy dalej
        System.err.println("Brak środków: " + e.getMessage());
        // wyjątek obsłużony, nie propagujemy dalej
    }
    // jeśli InvalidAmountException zostanie rzucony, poleci dalej
}
```

- Można łączyć wiele wyjątków połączonymi blokami:

```
catch (InsufficientFundsException | InvalidAmountException e) { ... }
```

- Lub używając typu nadrzędnego

```
catch (BankException e) { ... }
```

catch w C#

- Kolejne bloki mają szansę na przechwycenie pasującego wyjątku:

```
try {  
    // kod mogący rzucić wyjątek  
}  
catch (InsufficientFundsException e) {  
    Console.WriteLine($"Brak środków: {e.Message}");  
}  
catch (AccountNotFoundException e) {  
    Console.WriteLine($"Nie odnaleziono konta: {e.Message}");  
}
```

- Można użyć `catch (Exception)` do przechwycenia wszystkich wyjątków.
- Albo `catch` bez parametru (tracimy jednak obiekt wyjątku)
- Można dodać warunek filtrujący:

```
catch (Exception e) when (e is ArgumentException || e is InsufficientFundsException)  
{  
    Console.WriteLine($"Błąd operacji: {e.Message}");  
}
```

Obsługa błędów

Co możemy zrobić, gdy już złapiemy wyjątek?

- **Zapisanie szczegółów błędu** do logu systemowego, pliku, bazy danych lub systemu monitoringu.

```
logger.error("Błąd podczas przetwarzania płatności", e);
```

- **Wyświetlenie zrozumiałego komunikatu użytkownikowi** – bez technicznych szczegółów.

```
System.out.println("Wystąpił problem z wypłatą. Spróbuj ponownie.");
```

- **Wycofanie skutków rozpoczętej operacji (rollback)** – np. transakcji bazodanowej, zapisu pliku.

```
database.rollback();
```

- **Użycie wartości domyślnej lub kontynuacja działania** – o ile to możliwe.
- **Ponowne rzucenie wyjątku (rethrow)** – jeśli nie możemy obsłużyć błędu – przekazujemy go dalej.

```
throw e; // lub throw; w C++/C#
```

- **Zgłoszenie innego (opakowanego) wyjątku** – Zamieniamy niski poziom (np. `IOException`) na wyjątek domenowy.

```
throw new PaymentProcessingException("Nie udało się zrealizować płatności", e);
```

Obiekt wyjątku

- Obiekt wyjątku to nośnik informacji o błędzie, który został rzucony i złapany.
- Złapanie wyjątku to nie tylko odebranie sygnału o wystąpieniu błędu, ale pełnego obiektu, który zawiera:
 - **typ błędu** (np. `NullPointerException`, `IOException`, `std::invalid_argument`)
 - **opis błędu** (komunikat)
 - **miejsce wystąpienia błędu** (np. *stack trace*, nazwa metody)
 - opcjonalnie: przyczyna (*cause / inner exception*)
 - inne dane (np. pola własne, kod błędu, ID operacji)

Stack trace (ślad stosu) to automatycznie generowana lista wywołań funkcji/metod, które prowadziły do miejsca, w którym wystąpił wyjątek lub błąd.

Przykład (Java):

```
try {
    // ...
} catch (Exception e) {
    System.out.println("Typ: " + e.getClass().getName());
    System.out.println("Komunikat: " + e.getMessage());
    e.printStackTrace(); // pełny stack trace
}
```

Obsługa wyjątków – dobre i złe praktyki

- Nie należy zostawiać pustych bloków `catch` – jeśli nic nie robimy z wyjątkiem, należy przynajmniej go zalogować – tłumienie wyjątków utrudnia diagnozowanie błędów.

```
catch (BankException e) { }
```

- Łącuchowanie wyjątków (*exception chaining*) – opakowanie jednego wyjątku w inny nie powinno powodować utraty oryginalnej przyczyny błędu (*cause*)

```
catch (SQLException e) {  
    // konwertujemy wyjątek niskopoziomowy na domenowy  
    throw new AccountAccessException(accountId, e);  
}
```

- Jeśli rzucamy wyjątek dalej (*rethrow*), należy zachować kontekst.

```
catch (Exception e) {  
    LogError(e, "...");  
    throw; // lub throw e; w Javie  
}
```

Blok `finally`

- W Javie i C# instrukcja `try-catch-finally` lub `try-finally` pozwala dodać blok kodu, który wykonuje się **zawsze** – niezależnie od tego, czy w bloku `try` wystąpił wyjątek czy też nie.

```
try {  
    // kod, który może rzucić wyjątek  
}  
catch (ExceptionType e) {  
    // obsługa wyjątku  
}  
finally {  
    // kod porządkowy  
}
```

```
try {  
    // kod, który może rzucić wyjątek  
}  
finally {  
    // kod porządkowy  
}
```

- Służy głównie do zwalniania zasobów i wykonywania czynności porządkowych, które muszą się wykonać niezależnie od przebiegu operacji.
- Blok `finally` wykona się **zawsze** – niezależnie od:
 - wystąpienia wyjątku,
 - złapania lub niezłapania wyjątku,
 - `return` w `try` lub `catch`.
- Typowe zastosowanie: zamykanie strumieni (`close()`), połączeń, plików, transakcji.

Użycie `finally` do zwalniania zasobów

- Wyciek zasobów to nie tylko kwestia zwalniania pamięci, ale również:
 - zamknięcie pliku
 - zamknięcie połączenia z bazą danych
 - zwolnienie strumienia danych
- Blok `finally` gwarantuje wykonanie kodu zwalnającego zasoby.

```
BufferedReader reader = null;
try {
    reader = new BufferedReader(new FileReader(filename));
    String line = reader.readLine();
    System.out.println(line);
} catch (IOException e) {
    System.err.println("Błąd: " + e.getMessage());
} finally {
    // Zawsze się wykona, nawet jeśli wystąpi wyjątek powyżej
    if (reader != null) {
        try {
            reader.close();
        } catch (IOException e) {
            // nie poowinniśmy pozwalać na wyrzucenie wyjątków z finally
            System.err.println("Błąd zamykania pliku");
        }
    }
}
```

Zwalnianie zasobów - try-with-resources (Java)

- To nowocześniejszy i wygodniejszy sposób zarządzania zasobami.
- Konstrukcja wprowadzona w **Java 7**.
- Automatycznie zamyka zasoby po zakończeniu bloku `try` (bez potrzeby jawnego użycia `finally`).

```
try (BufferedReader reader = new BufferedReader(new FileReader("plik.txt"))) {  
    String line = reader.readLine();  
    System.out.println("Odczytano: " + line);  
} catch (IOException e) {  
    System.err.println("Błąd odczytu pliku: " + e.getMessage());  
}
```

- Java automatycznie wywołuje metodę `close()`, nawet jeśli wystąpi wyjątek.
- Działa z dowolnym obiektem implementującym interfejs `AutoCloseable` (np. `BufferedReader`, `InputStream`, `Connection`, `Scanner`, `PreparedStatement`).
- Umożliwia zadeklarowanie wielu zasobów jednocześnie:

```
try (  
    BufferedReader reader = ...;  
    BufferedWriter writer = ...  
)
```

Zwalnianie zasobów – `using` (C#)

- W C# do automatycznego zarządzania zasobami służy słowo kluczowe `using`.
- Zapewnia automatyczne zwolnienie zasobów implementujących interfejs `IDisposable` (wywołuje metodę `Dispose()`).

```
using (var reader = new StreamReader("plik.txt")) {  
    string line = reader.ReadLine();  
    Console.WriteLine(line);  
}  
// reader.Dispose() został automatycznie wywołany
```

- Od C# 8 można używać `using` bez nawiasów klamrowych.

```
using var reader = new StreamReader("plik.txt");  
Console.WriteLine(reader.ReadLine());
```

- Można deklarować wiele zasobów jednocześnie:

```
using (var reader = ..., var writer = ...) { ... }
```

Zwalnianie zasobów – **RAII** (C++)

- **RAII** – *Resource Acquisition Is Initialization* – oznacza to, że zasoby (pliki, pamięć, połączenia) są wiązane z życiem obiektów.
- Zasób jest przydzielany w konstruktorze i zwalniany w destruktorze.

```
void readFile() {  
    std::ifstream file("plik.txt"); // konstruktor otwiera plik  
    if (!file.is_open()) {  
        throw std::runtime_error("Nie można otworzyć pliku");  
    }  
  
    std::string line;  
    std::getline(file, line);  
    riskyOperation(line); // nawet jeśli operacja rzuci wyjątek,  
                          // destruktor obiektu file i tak zostanie wywołany  
}  
// tutaj file zostaje automatycznie zamknięty przez destruktor
```

- `std::ifstream` sam zamyka plik w destruktorze – nie trzeba robić tego ręcznie

Nieobsłużone wyjątki

- Jeśli **nie złapiemy wyjątku**, jego obsługa przechodzi do **mechanizmu awaryjnego danego języka** – co **zwykle prowadzi do zakończenia programu z błędem** oraz wyświetlenia ***stack trace*** (śladu stosu).

Przykład (C#):

```
static void Main() {  
    Divide(5, 0); // nie łapiemy wyjątku  
}
```

Output:

```
System.DivideByZeroException: Attempted to divide by zero.  
   at Program.Divide(Int32 a, Int32 b)  
   at Program.Main()
```

- Nieobsłużony wyjątek propaguje się w górę stosu aż do `Main()`.
- Jeśli nigdzie nie zostanie złapany, to program zostaje przerwany i zgłaszany jest błąd wykonania (*stack trace* trafia na konsolę).

Rzucanie wyjątków w konstruktorze

- Rzucanie wyjątku w konstruktorze ma sens i zastosowanie, gdy:
 - obiekt **nie może zostać poprawnie zainicjalizowany** (np. brak pliku, złe dane),
 - chcemy **zatrzymać tworzenie obiektu**, zanim będzie gotowy do użycia.
- Jeśli konstruktor rzuci wyjątek:
 - obiekt **nie zostanie** utworzony,
 - uruchamia się *stack unwinding* – w C++ **destruktorzy już skonstruowanych pól** zostaną wywołane,
 - w C++ destruktor klasy **nie zostanie wywołany** (bo nie było udanej konstrukcji).
- Jest to bezpieczne, jeśli prawidłowo zarządzamy zasobami.
 - W Javie/C# zwolnieniem zasobów zajmie się GC (*garbage collector*),
 - ale może wystąpić wyciek np. pliku, jeśli nie zamkniemy go wcześniej.
- Z kolei **rzucanie wyjątków w destruktorze jest niedozwolone. ❌**
 - Destructory są często wywoływane podczas obsługi wyjątku (*stack unwinding*).
 - Jeśli w tym samym czasie inny wyjątek zostanie rzucony z destruktora, program **zakończy się przez**
`std::terminate()`.
 - To może prowadzić do trudnych do znalezienia błędów.

Specyfikowanie wyjątków w dziedziczeniu (Java)

Zasady dotyczą *checked exceptions* (wymagających obsługi lub jawnej deklaracji).

Podczas przesłaniania (*overriding*) metody w klasie potomnej, możemy:

- rzucać **te same wyjątki** co metoda bazowa,
- **mniej** (lub rzucać wyjątki bardziej szczegółowe),
- **nie rzucać żadnych** (czyli zrezygnować z `throws`).

Nie możemy natomiast dodać **nowych, szerszych (bardziej ogólnych)** *checked exceptions*.

Przykład:

```
class BankOperationException extends Exception { }
class InsufficientFundsException extends BankOperationException { }

class Bank {
    public void withdraw(double amount) throws BankOperationException {
        // ...
    }
}

class SecureBank extends Bank {
    @Override
    public void withdraw(double amount) throws InsufficientFundsException {
        // OK: bardziej szczegółowy wyjątek
    }
}
```

Specyfikowanie wyjątków w dziedziczeniu (Java)

- Z czego wynika ta zasada?
- Metodę `withdraw()` możemy wywołać z referencji klasy bazowej.

```
Bank bank = new SecureBank();
try {
    bank.withdraw(1200.0);
} catch (BankOperationException e) {
    e.printStackTrace();
}
```

- Kod operuje na typie bazowym i spodziewa się wyjątków zadeklarowanych w sygnaturze metody w klasie `Bank`.
- Nie możemy otrzymać nowego wyjątku, którego się nie spodziewamy i nie wiemy jak obsłużyć.
- Nie ma natomiast problemu, jeśli implementacja z klasy pochodnej rzuca mniej wyjątków lub bardziej specyficzne.
- W **konstruktorach** jest na odwrót – w klasie pochodnej możemy dodać lub uogólnić wyjątki, ale nie usunąć.
 - Wynika to z tego, że konstruktor klasy pochodnej wywołuje konstruktor klasy bazowej – a zatem może rzucać wszystkie wyjątki, które on rzucał.
- **W razie wątpliwości** – najlepiej powtórzyć specyfikację wyjątków z klasy bazowej.